

REPORTE TAREA 1

ALGORITMOS Y COMPLEJIDAD

«Más allá de la notación asintótica: Análisis experimental de algoritmos de ordenamiento y multiplicación de matrices.»

Martín González Asenjo

11 de septiembre de 2026

23:55

1. Introducción

Estudiar algoritmos no es solo mirar una fórmula y quedarse tranquilo; el punto de este informe es ver qué pasa cuando esa fórmula se enfrenta a datos reales. Aquí se trabajaron dos problemas clásicos: ordenar arreglos de una dimensión y multiplicar matrices cuadradas. Se pusieron a prueba MergeSort, QuickSort, PatienceSort y `std::sort` por un lado, y los métodos Naive y Strassen por el otro, comparando sus tiempos y su uso de memoria contra lo que la teoría predice [4, 3]. La pregunta de fondo es hasta qué punto las mediciones reales calzan con el comportamiento asintótico esperado, y qué tanto pesan factores como el tamaño de la entrada, su estructura y cómo están distribuidos los datos.

2. Entorno experimental

Todo se corrió en un equipo con Windows, usando WSL2 (Ubuntu), g++ con las flags `-std=c++17 -O2 -Wall -Wextra`, más Python 3, NumPy y Matplotlib para el procesamiento y los gráficos. Los programas registran tiempo en microsegundos y memoria residente en KB, esto último vía `getrusage`. Cada algoritmo trabajó sobre una copia idéntica del input y su resultado se verificaba antes de guardar cualquier medición.

Para el sorting se probaron los tamaños $n \in \{10, 10^3, 10^5\}$ combinando los tres tipos de orden con los dominios $D1$ y $D7$, tres muestras por combinación, y además el caso $n = 10^7$ aleatorio, corrido completo (muestras a, b, c) en $D7$ y de forma parcial (solo muestra a) en $D1$. En total se completaron los cuatro algoritmos en 57 de las 72 entradas de sorting, más una entrada parcial (10^7 , aleatorio, $D1$, mues-

tra *a*) donde solo alcanzaron a medirse MergeSort y QuickSort antes de que se cerrara el plazo de entrega. Los 14 casos que quedaron sin correr en $n = 10^7$ (dos muestras aleatorias en *D1* más las doce combinaciones ascendente/descendente) se dejaron fuera a propósito: con pivote fijo en el primer elemento, QuickSort cae en $O(n^2)$ apenas la entrada está ordenada o tiene muchos valores repetidos, y a PatienceSort le pasa algo parecido porque recorre sus pilas de forma lineal — en ambos casos el tiempo de ejecución se vuelve impracticable. Para las matrices se usó $n \in \{2^4, 2^6, 2^8, 2^{10}\}$, cruzando los tipos dispersa, diagonal y densa con los dominios *D0* y *D10*, tres muestras cada uno, lo que da 72 pares de matrices en total. Todos los datos se generan con los scripts que están en `code/sorting/scripts/` y `code/matrix_multiplication/scripts/`.

Las implementaciones de ordenamiento siguen la descripción de MergeSort y QuickSort que da Cormen et al. [2], apoyada además en las referencias de MergeSort [7] y QuickSort [9]. PatienceSort se construyó a partir de la descripción del algoritmo y su vínculo con las subsecuencias crecientes [1, 8]. Para la multiplicación de matrices, la versión Naive sigue el esquema clásico de tres ciclos anidados de Cormen et al. [2]; la versión Strassen usa la reducción a siete multiplicaciones que propuso el propio Strassen [6], tomando como referencia la implementación de Rangelak [5]. Estas fuentes también quedaron anotadas al inicio de cada archivo correspondiente, dentro de `code/sorting/algorithms/` y `code/matrix_multiplication/algorithms/`.

3. Resultados y Análisis

Las tablas que siguen muestran el promedio sobre las muestras disponibles para cada tamaño. Tiempos en microsegundos, memoria en KB.

Cuadro 1: Tiempo promedio de sorting por tamaño (microsegundos).

n	MergeSort	QuickSort	PatienceSort	<code>std::sort</code>
10	0.78	0.00	1.67	0.44
10^3	46.94	173.50	452.61	11.28
10^5	5315.83	1434692.50	4711136.44	1518.11
10^7	1114715.50	666462.33*	390309072.33	704833.00

*Promedio de las tres muestras *D7* (aleatorio). Se excluye a propósito la muestra *D1, a* de $n = 10^7$: ahí QuickSort tardó **3 045 046 767 us (~50.8 minutos)**, casi tres órdenes de magnitud más que en *D7*.

Con solo 10 valores distintos, tener el pivote fijo (siempre el primer elemento) genera particiones desbalanceadas de manera sistemática, no por casualidad, y el tiempo medido sigue casi al pie de la letra la curva de referencia $O(n^2)$ que se ve en la Figura 1. De hecho es el caso más claro de todo el experimento: la cota superior del peor caso deja de ser algo teórico y se vuelve algo que uno observa directamente en los números.

La Tabla 1 deja claro que `std::sort` se mantiene competitivo de forma consistente. En 10^5 , tanto QuickSort como PatienceSort empeoran, sobre todo frente a entradas con valores repetidos o parcial-

mente ordenadas — un problema que se agrava mucho más en 10^7 para QuickSort dentro del dominio $D1$ (ver la nota anterior). Mirando solo $D7$, PatienceSort tardó en promedio unos 390 segundos en $n = 10^7$, muy por encima del $O(n \log n)$ que sí logran `std::sort` y MergeSort. Las Figuras 1 y 2 muestran estas tendencias en escala logarítmica.

Cuadro 2: Memoria residente promedio registrada en sorting (KB).

n	MergeSort	QuickSort	PatienceSort	<code>std::sort</code>
10	0	0	0	0
10^3	0	0	0	0
10^5	0	0	0	0
10^7	19520.00	0	38566.67	0

La memoria se estimó como la diferencia de `ru_maxrss` antes y después de cada algoritmo. Un valor de cero significa que no se detectó un aumento del máximo residente en esa ejecución particular, no que el algoritmo no use memoria adicional.

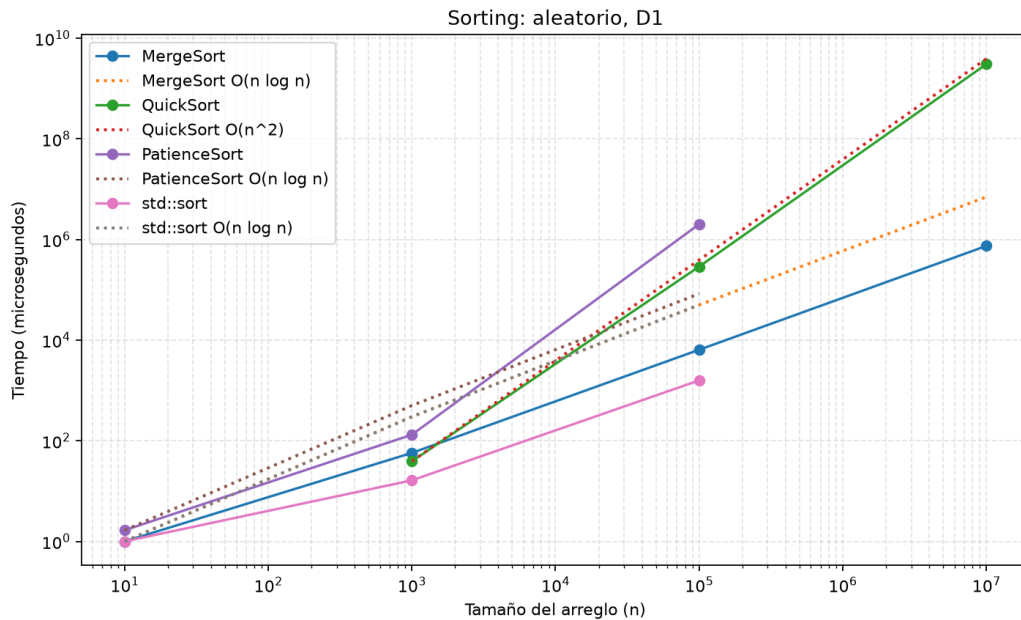


Figura 1: Sorting aleatorio en dominio $D1$.

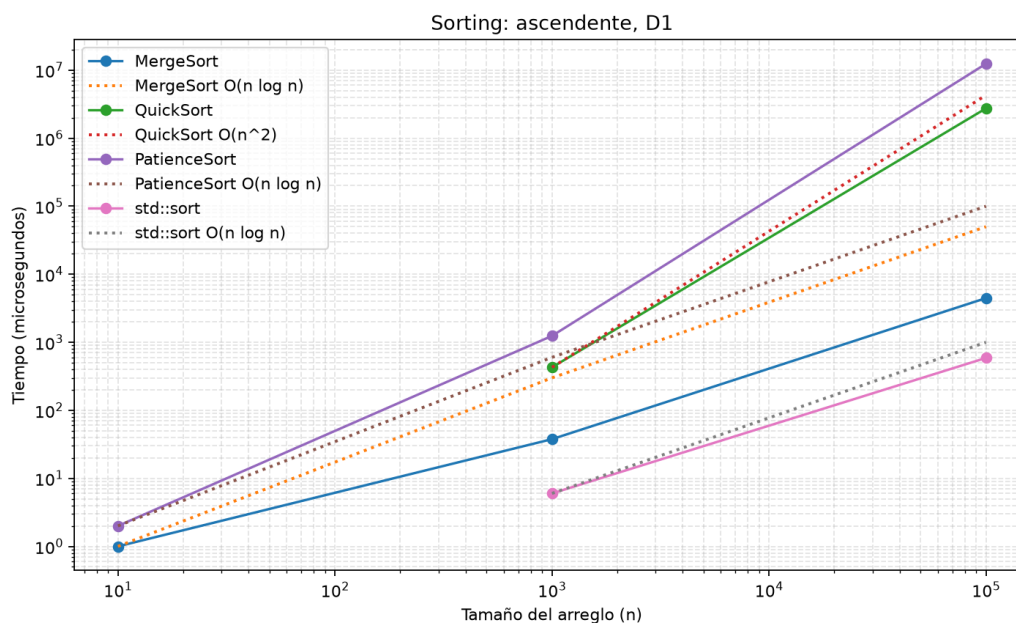


Figura 2: Sorting ascendente en dominio $D1$.

Cuadro 3: Tiempo promedio de multiplicación de matrices.

n	Naive	Strassen
16	1.72	519.11
64	92.50	23323.11
256	7236.06	1150298.44
1024	764579.33	55713463.44

En la Tabla 3 se ve que Strassen termina siendo más caro en este experimento, principalmente por sus constantes altas y por las asignaciones y operaciones extra que arrastra. La diferencia se nota bien en las Figuras 3 y 4; en todos los casos procesados, ambas implementaciones dieron matrices idénticas.

Cuadro 4: Memoria residente promedio registrada en matrices.

n	Naive	Strassen
16	0	0
64	0	0
256	0	0
1024	0	1315.56

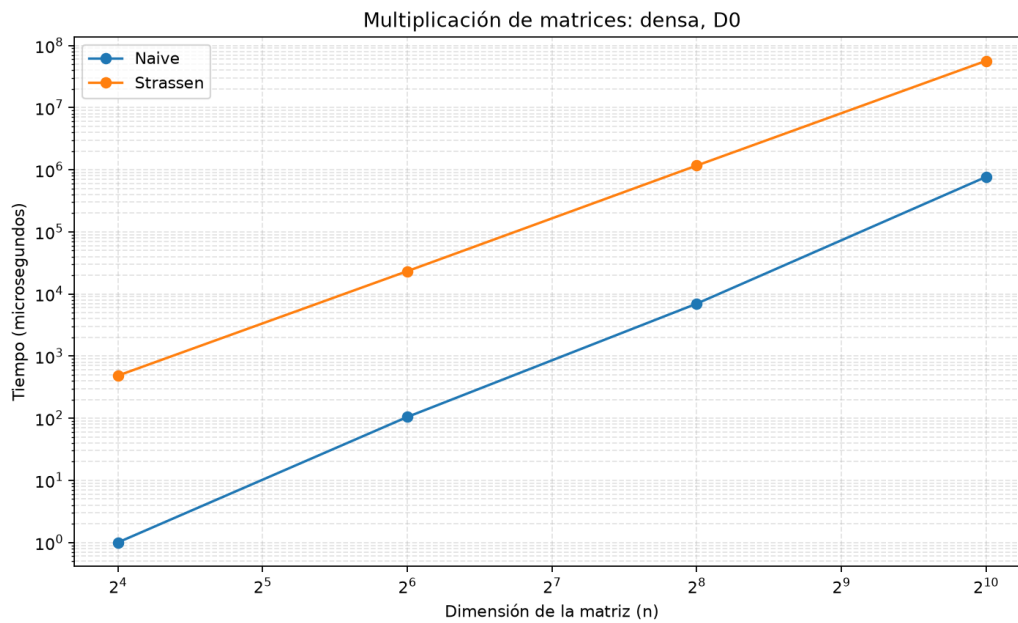


Figura 3: Multiplicación de matrices densas con dominio $D0$.

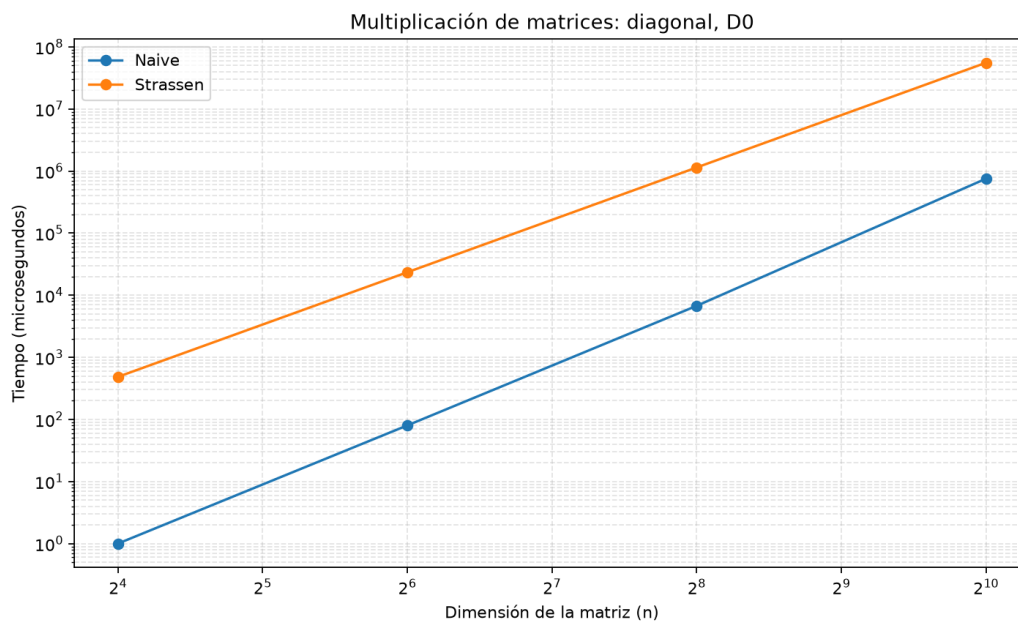


Figura 4: Multiplicación de matrices diagonales con dominio $D0$.

Las visualizaciones se generan automáticamente corriendo `make plot` en cada directorio. Las figuras usadas están en [code/sorting/data/plots/](#) y [code/matrix_multiplication/data/plots/](#).

4. Conclusiones

En general, el desempeño real depende tanto del tamaño de la entrada como de cómo están distribuidos los datos. En `std::sort` fue el más rápido en los tamaños chicos y medianos, mientras que QuickSort y PatienceSort mostraron ser particularmente vulnerables a entradas con muchos valores repetidos o con cierto orden inicial. El caso más contundente de todo el informe fue QuickSort en $n = 10^7$ con dominio $D1$ (pocos valores distintos): al usar pivote fijo en el primer elemento, su tiempo se disparó a ~50.8 minutos siguiendo casi exactamente la curva teórica $O(n^2)$, mientras que en $D7$ (valores mayormente únicos) el mismo algoritmo terminó en menos de un segundo. Es la prueba más clara de que la cota de peor caso no es solo un límite teórico: aparece en la práctica en cuanto la entrada tiene la estructura justa para gatillarla. Dentro de $D7$, PatienceSort fue el más costoso en 10^7 (~390 segundos en promedio), comparado con menos de un segundo de `std::sort`. En cuanto a matrices, Strassen resultó más lento que Naive en todas las dimensiones probadas, aunque ambos entregaron resultados idénticos; esto deja en evidencia que tener una mejor complejidad asintótica no garantiza ninguna ventaja real cuando el tamaño del problema todavía deja que dominen las constantes y el manejo de memoria.

Referencias

- [1] David Aldous y Persi Diaconis. «Longest increasing subsequences: from patience sorting to the Baik-Deift-Johansson theorem». En: *Bulletin of the American Mathematical Society* 36.4 (1999), págs. 413-432. DOI: [10.1090/S0273-0979-99-00796-X](https://doi.org/10.1090/S0273-0979-99-00796-X). URL: <https://doi.org/10.1090/S0273-0979-99-00796-X>.
- [2] Thomas H. Cormen et al. *Introduction to Algorithms*. 3.^a ed. MIT Press, 2009. URL: <https://mitpress.mit.edu/9780262033848/introduction-to-algorithms/>.
- [3] Jeff Erickson. *Algorithms*. Jun. de 2019. ISBN: 978-1-792-64483-2.
- [4] Donald E. Knuth. *The art of computer programming, volume 3: (2nd ed.) sorting and searching*. USA: Addison Wesley Longman Publishing Co., Inc., 1998. ISBN: 0201896850.
- [5] Rangelak. *Strassen-Matrix-Multiplication*. 2026. URL: <https://github.com/rangelak/Strassen-Matrix-Multiplication/blob/master/strassen.cpp> (visitado 11-09-2026).
- [6] Volker Strassen. «Gaussian elimination is not optimal». En: *Numerische Mathematik* 13 (1969), págs. 354-356. DOI: [10.1007/BF02165411](https://doi.org/10.1007/BF02165411). URL: <https://doi.org/10.1007/BF02165411>.
- [7] Wikipedia contributors. *Merge sort*. 2026. URL: https://en.wikipedia.org/wiki/Merge_sort (visitado 11-09-2026).
- [8] Wikipedia contributors. *Patience sorting*. 2026. URL: https://en.wikipedia.org/wiki/Patience_sorting (visitado 11-09-2026).
- [9] Wikipedia contributors. *Quicksort*. 2026. URL: <https://en.wikipedia.org/wiki/Quicksort> (visitado 11-09-2026).